

Implementation Note

Numerical MRA-DMRG Orbitals with erfc Transcorrelation

A code-independent guide to the representation, workflow, and operators

Case study helium atom in cc-pVDZ

Version 2.12
24 July 2026

Abstract

This note explains the route from Gaussian molecular orbitals to optimized numerical multiresolution orbitals, and then to a fixed-orbital erfc transcorrelated Hamiltonian. The first part follows the direct real-space orbital framework of Valeev *et al.* and its DMRG extension by Nibbi *et al.* [1, 2]. The second part uses the Jastrow similarity transform of Cohen *et al.* and the erfc correlation factor of Giner [3, 4]. The presentation is independent of a particular source tree. It defines what an adaptive MRA tree stores, shows a complete one-dimensional projection and reconstruction, and derives the three-dimensional operations used to form the ordinary Coulomb and transcorrelated integrals. The molecular orbitals remain adaptive three-dimensional MRA trees; only the radial transcorrelated kernels are converted to short Gaussian sums for separated convolution.

Contents

1	Scope and computational separation	1
I	Numerical MRA-DMRG orbital generation	2
2	Orbital representation	2
2.1	Adaptive MRA trees and MO projection	2
2.2	Back-projection to a Gaussian basis	6
3	Plain MRA integrals	6
3.1	One-electron matrix	6
3.2	Two-electron tensor	8
4	MRA-DMRG orbital optimization	9
4.1	Finite second-quantized problem	9
4.2	Lagrangian derivative and orbital update	10
4.3	Output	12
II	Fixed-orbital erfc transcorrelation	13
5	Similarity transform and erfc Jastrow factor	13
6	Radial kernels and Gaussian separation	14
6.1	Giner’s Gaussian-orbital route and the present MRA route	14
6.2	Exact integral identities and finite GL sums	15
6.3	Why the present MRA route uses Gaussian sums	16
7	Operator-level construction of K_1, K_2, K_3, and L	17
7.1	K_1 : non-Hermitian drift term	17
7.2	K_2 : direct Laplacian convolution	18
7.3	K_3 : squared-gradient scalar term	18
7.4	VAMPyR-style bridge from Gaussian sums to MRA fields	19
7.5	L : factored three-electron term	19
8	Capabilities and boundaries	20

1 Scope and computational separation

The workflow contains two logically separate calculations.

1. A plain-Hamiltonian MRA-DMRG calculation optimizes a finite set of molecular orbitals in real space and stores each orbital as an adaptive tree.
2. A fixed-orbital transcorrelated calculation takes either these optimized trees or unoptimized PySCF orbitals projected into the same MRA world, builds the additional TC integrals, and solves the resulting non-Hermitian Hamiltonian.

The TC stage does not currently feed back into the orbital optimization. The Jastrow parameter is scanned rather than variationally optimized.

A separate plain-Hamiltonian regression test isolates the projection and integral boundary. It evaluates PySCF RHF MOs and their Gaussian-basis h and g , projects the same MOs into one MRA world, rebuilds the two integral tensors from the trees, and compares $\max|\Delta h|$, $\max|\Delta g|$, and the resulting plain-FCI energy. This check precedes both orbital optimization and all TC corrections.

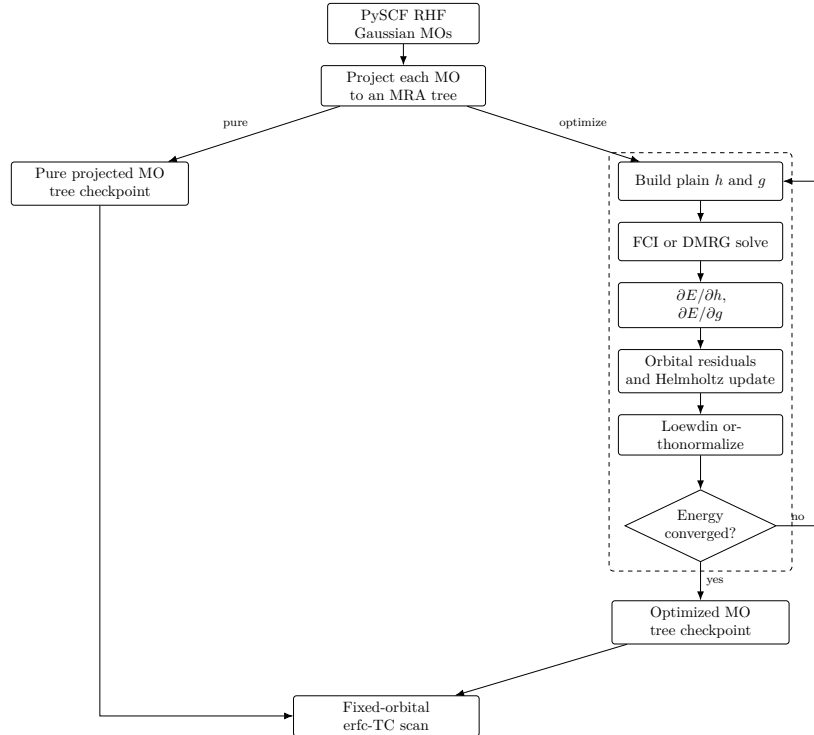


Figure 1: Complete computational flow. Both orbital families enter the TC stage as numerical molecular orbitals stored in a compatible MRA world.

Part I

Numerical MRA-DMRG orbital generation

2 Orbital representation

2.1 Adaptive MRA trees and MO projection

Each stored function is a three-dimensional *molecular orbital* $\phi_p(\mathbf{r})$, not a numerical atomic-orbital basis function. To make the tree representation explicit, first consider one dimension. Let $I_{n\ell} = [\ell 2^{-n}, (\ell + 1)2^{-n}]$. The scaling space

$$V_n^k = \{f : f|_{I_{n\ell}} \text{ is a polynomial of degree at most } k\} \quad (\text{P1.1})$$

contains independent local polynomials on the level- n dyadic cells. Normalized scaling functions may be written

$$\varphi_{j\ell}^n(x) = 2^{n/2} \varphi_j(2^n x - \ell), \quad j = 0, \dots, k. \quad (\text{P1.2})$$

In the VAMPyR interface, k is the value supplied to `LegendreBasis(order=k)`. MRCPMP then uses the $k + 1$ Legendre scaling functions $j = 0, \dots, k$, so a terminal three-dimensional node contains $(k + 1)^3$ scaling coefficients. Increasing k raises the degree of the local polynomial and the amount of work and storage per node; for a smooth function it can reduce the number of dyadic levels required. The value of k is fixed for a given MRA world. Adaptivity changes the number and location of cells, not the polynomial order from one cell to another.

The second numerical control is the VAMPyR/MRCPMP `prec` argument, denoted here by p_{tol} to distinguish it from an orbital index. Projection and operator application first compute local scaling coefficients and their multiwavelet details. MRCPMP compares the local wavelet norm with a level-scaled threshold proportional to p_{tol} . A cell whose detail remains above the threshold is split into dyadic children; a sufficiently small detail is retained at the coarser level and can be removed by cropping. With the usual relative precision mode the threshold is also scaled by the function norm; an absolute-precision mode is available separately. Decreasing p_{tol} therefore normally produces a larger and possibly deeper tree. It is neither a grid spacing nor a percentage, and it does not guarantee the same pointwise error at every coordinate. Results should be converged with respect to both k and p_{tol} , which are calculation settings rather than fixed properties of the He/cc-pVDZ case study.

The orthogonal complement of V_n^k inside V_{n+1}^k is the multiwavelet space W_n^k :

$$V_{n+1}^k = V_n^k \oplus W_n^k. \quad (\text{P1.3})$$

Thus a finest-level projection has the equivalent multiresolution form

$$P_N f = \sum_j s_j^{(0)} \varphi_{j0}^0 + \sum_{n=0}^{N-1} \sum_{\ell,j} d_{j\ell}^{(n)} \psi_{j\ell}^n. \quad (\text{P1.4})$$

The s 's are scaling coefficients and the d 's are detail (multiwavelet) coefficients. Equation (P1.4), rather than point samples on a uniform grid, is the information represented by the tree.

In three dimensions the basis functions are tensor products. A parent box has eight dyadic children. The orthogonal transform from the eight child scaling tensors produces one parent scaling block and seven detail blocks, labelled by the nonzero combinations of scaling/wavelet character along x, y, z . Every block contains $(k + 1)^3$ coefficients. This is a *local transform*:

the globally compressed tree retains the coarse scaling representation and only the significant detail blocks at active nodes. It is therefore misleading to imagine that every node in the final tree independently stores one scaling and seven detail blocks.

Worked one-dimensional projection and reconstruction

Consider the cusp-like function

$$f(x) = e^{-4|x-1/3|}, \quad 0 \leq x \leq 1, \quad (\text{P1.5})$$

whose cusp does not coincide with a boundary at any finite dyadic level. First choose $k = 0$ with three uniform dyadic levels. The normalized finest-level scaling function is $\varphi_\ell^3(x) = \sqrt{8} \mathbf{1}_{I_{3\ell}}(x)$, so the eight projection coefficients are

$$c_\ell^{(3)} = \langle \varphi_\ell^3, f \rangle = \sqrt{8} \int_{\ell/8}^{(\ell+1)/8} f(x) \, dx. \quad (\text{P1.6})$$

Numerically,

ℓ	0	1	2	3	4	5	6	7
$c_\ell^{(3)}$	.120916	.199357	.308996	.235512	.142845	.086640	.052550	.031873

For the Haar ($k = 0$) transform, every sibling pair obeys

$$s = \frac{c_L + c_R}{\sqrt{2}}, \quad d = \frac{c_L - c_R}{\sqrt{2}}, \quad c_L = \frac{s + d}{\sqrt{2}}, \quad c_R = \frac{s - d}{\sqrt{2}}. \quad (\text{P1.7})$$

Applying the forward transform recursively gives

$$\begin{aligned} \mathbf{d}^{(2)} &= (-.055466, .051961, .039743, .014621), \\ \mathbf{d}^{(1)} &= (-.112118, .072531), \\ d_0^{(0)} &= .194763, \quad s_0^{(0)} = .416730. \end{aligned}$$

The tree stores $s_0^{(0)}$ together with these level-dependent details. For example, the complete inverse path to the third fine cell, which contains the cusp, is

$$s_0^{(1)} = \frac{s_0^{(0)} + d_0^{(0)}}{\sqrt{2}}, \quad s_1^{(2)} = \frac{s_0^{(1)} - d_0^{(1)}}{\sqrt{2}}, \quad c_2^{(3)} = \frac{s_1^{(2)} + d_1^{(2)}}{\sqrt{2}} = .308996.$$

Repeating this for all branches recovers the eight $c_\ell^{(3)}$ to floating-point precision. The reconstructed function is the projection $P_3^{(0)}f$, not the analytic function f . Its L^2 error is 6.09×10^{-2} , and the error at the cusp is 1.26×10^{-1} . In an adaptive calculation a detail block with norm below the requested threshold can be omitted.

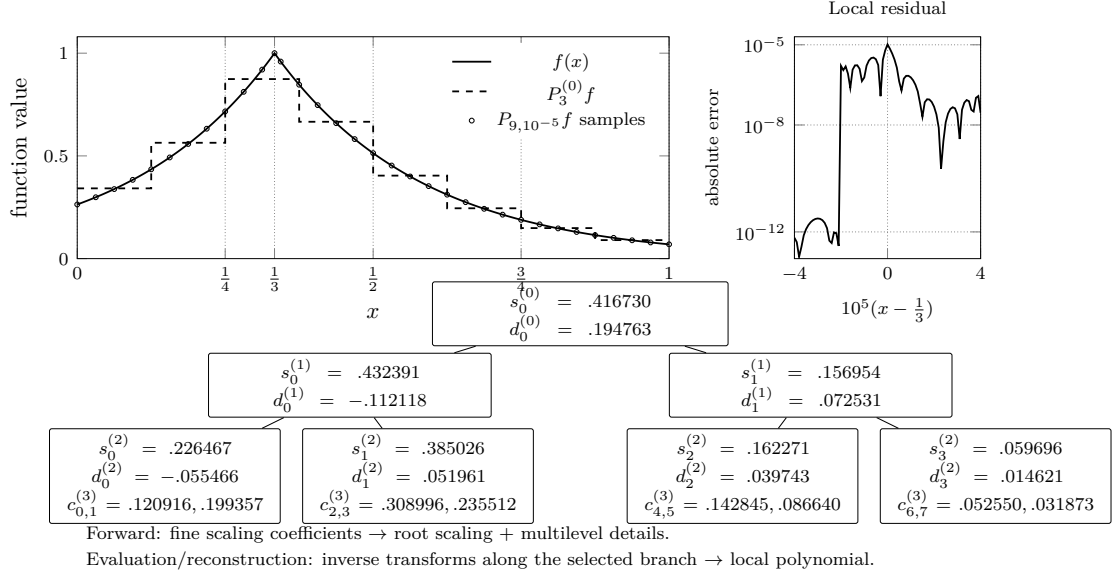


Figure 2: The same off-dyadic cusp represented at two resolutions. The $k = 0$, three-level projection is visibly piecewise constant, and its complete coefficient tree is shown below. Samples from the adaptive $k = 9$, $p = 10^{-5}$ projection lie on the analytic curve at plotting resolution; the logarithmic panel resolves its error near the cusp.

The same function was also projected with the production settings $k = 9$ and $p = 10^{-5}$. This one-dimensional tree contained 27 nodes, of which 14 were terminal. Direct quadrature against the analytic function gave

$$\|P_{9,10^{-5}}f - f\|_2 = 1.68 \times 10^{-8}, \quad |P_{9,10^{-5}}f(1/3) - f(1/3)| = 1.03 \times 10^{-5}.$$

The L^2 value agrees with a same-order $p = 10^{-11}$ reference tree. The parameter p controls coefficient refinement and truncation; it is not a strict pointwise error bound at every Cartesian coordinate.

From a Gaussian MO to an MRA MO

PySCF first supplies RHF molecular orbitals in a spherical Gaussian AO basis. To make the code path explicit, let a index primitives in contracted AO A . For the present He/cc-pVDZ calculation, the primitive passed to `vp.GaussFunc` is the unnormalized Cartesian monomial

$$g_{Aa}(\mathbf{r}) = X_{\ell_A m_A}(\mathbf{r} - \mathbf{R}_A) \exp[-\alpha_{Aa}|\mathbf{r} - \mathbf{R}_A|^2], \quad (\text{P1.8})$$

where $X_{00} = 1$ for an s function and X_{1m} is x , y , or z for a p function. In the implementation, `bas_ctr_coeff(shell)` multiplied by `gto_norm( $\ell$ , exponents)` gives the radial coefficient that multiplies this monomial. Because `GaussFunc` does not include the real-spherical angular normalization, the code then inserts the exact factors

$$N_s = \frac{1}{2\sqrt{\pi}}, \quad N_p = \sqrt{\frac{3}{4\pi}}. \quad (\text{P1.9})$$

The contracted spherical AO and the PySCF RHF MO are consequently

$$\chi_A(\mathbf{r}) = \sum_a d_{Aa}^{\text{rad}} N_{\ell_A} g_{Aa}(\mathbf{r}), \quad \phi_p(\mathbf{r}) = \sum_A C_{Ap} \chi_A(\mathbf{r}), \quad (\text{P1.10})$$

where C_{Ap} is a column of `mo_coeff`.

The implementation forms the same hierarchy directly in the MRA:

$$T_{Aa} = P_{\text{MRA}}[g_{Aa}], \quad T_A^{\text{AO}} = \sum_a d_{Aa}^{\text{rad}} N_{\ell_A} T_{Aa},$$

$$T_p^{\text{MO}} = \sum_A C_{Ap} T_A^{\text{AO}} = P_{\text{MRA}} \left[\sum_A C_{Ap} \chi_A \right] = P_{\text{MRA}}[\phi_p]. \quad (\text{P1.11})$$

The last equality follows from the linearity of P_{MRA} . Projecting each unique primitive once and then taking the AO and MO linear combinations is therefore the projection of the full LCAO, not a grid interpolation of the MO. The corresponding code objects are named `primitive_trees`, `ao_trees`, and `mo_trees`.

Values at local tensor-product quadrature points are transformed into scaling coefficients; the multiwavelet transform measures the details. Cells with significant detail are split into eight children, while smooth regions remain coarse. This adaptive cell quadrature is internal to `ScalingProjector`; it is not a fixed global grid of AO or MO values.

Finite projection and cropping tolerances can leave the projected MOs slightly nonorthogonal. After the MO trees are built, a separate Löwdin step forms

$$S_{pq} = \langle T_p^{\text{MO}} | T_q^{\text{MO}} \rangle, \quad T_p^{\text{orth}} = \sum_q T_q^{\text{MO}} (S^{-1/2})_{qp}. \quad (\text{P1.12})$$

The pure checkpoint stores these projected, orthonormalized MO trees. The optimized checkpoint stores the MO trees returned by the orbital optimizer. Primitive and AO trees are temporary initialization objects and are not persisted. Neither checkpoint is a finite Gaussian AO coefficient vector. The subsequent optimization does not perform another AO-to-MO transformation; it updates the real-space MO trees directly.

The exact shortcut above currently supports s and p shells. Starting with d , a real-spherical AO is generally a linear combination of several Cartesian monomials. Extending the analytic route therefore requires the exact PySCF Cartesian-to-spherical transformation, including its ordering, normalization, and phase conventions. Unsupported higher-angular-momentum shells are rejected rather than assigned an assumed convention. A general pointwise PySCF or VeloxChem MO evaluator remains an alternative projection source [5], but it is not the path used by this tutorial.

For a molecular s -like orbital, $\phi(\mathbf{r}) \sim e^{-Z|\mathbf{r}-\mathbf{R}_A|}$, large details occur near the nuclear cusp; nodes, bonds, and long-range tails create other local refinement patterns. A finite expansion in centered s -type Gaussians $\sum_m a_m e^{-\alpha_m |\mathbf{r}-\mathbf{R}_A|^2}$ is analytic and has zero radial derivative at the nucleus, so it cannot reproduce the nonzero Slater cusp exactly. Increasing Gaussian flexibility therefore requires multiple tight, valence, and diffuse exponents (and, in molecules, additional angular components). MRA instead assigns resolution locally; there is no universal number of Gaussians equivalent to a given adaptive tree.

At a Cartesian coordinate, the coordinate bits select an octree branch. Inverse multiwavelet transforms recover the terminal scaling tensor, and the local tensor-product polynomial is evaluated at the mapped coordinate. Projection, differentiation, multiplication, evaluation, and persistence can be expressed without assuming a particular software interface:

Pseudocode: analytic Gaussian AO-to-MO projection

INPUT: PySCF basis metadata, C_{Ap} , and MRA world (Ω, k, p)

for each unique primitive g_{Aa} :

```

primitive_trees[Aa] <- ScalingProjector(gAa)
for each contracted AO A:
  ao_trees[A] <-  $\sum_a d_{Aa}^{rad} N_{\ell_A}$ 
                    primitive_trees[Aa]
for each molecular orbital p:
  mo_trees[p] <-  $\sum_A C_{Ap}$  ao_trees[A]
projected_mos <- Lowdin(mo_trees)

for each projected or optimized MO  $\phi_p$ :
  ( $\partial_x \phi_p, \partial_y \phi_p, \partial_z \phi_p$ )
  <- differentiate the MO tree when required

 $\rho_{pq}$  <- multiply the compatible trees  $\phi_p^*$  and  $\phi_q$ 
 $\phi_p(x, y, z)$  <- follow one octree branch, reconstruct the
                    terminal tensor, and evaluate its local polynomial
SAVE: MO coefficient trees plus MRA-world metadata

```

These operations act directly on adaptive coefficient trees. They do not first sample the orbital on a global Cartesian grid.

The MRA world fixes the finite domain, polynomial order k , target precision, and boundary and operator conventions. Geometry and the chosen nuclear potential define the Hamiltonian. These settings must accompany a saved orbital tree so that the same adaptive function can be reconstructed.

2.2 Back-projection to a Gaussian basis

In principle, each optimized MRA molecular orbital can be projected onto a chosen Gaussian AO basis, yielding an approximate set of AO expansion coefficients for the molecular orbitals. With $S_{\mu\nu}^{\text{AO}} = \langle \chi_\mu | \chi_\nu \rangle$ and $b_{\mu p} = \langle \chi_\mu | \phi_p^{\text{MRA}} \rangle$, the least-squares coefficients satisfy

$$\sum_{\nu} S_{\mu\nu}^{\text{AO}} C_{\nu p} = b_{\mu p}. \quad (\text{P1.13})$$

However, the back-projection is not generally lossless. Reproducing local cusps, nodes, and diffuse tails may require many more Gaussian primitives, higher angular momenta, and both tight and diffuse exponents. Compact, local refinement is precisely the advantage of the MRA representation.

For a real-space downstream program, the most faithful interface is the tree plus an evaluator for $\phi_p(x, y, z)$ and its Cartesian derivatives. A PySCF checkpoint reconstructed through a finite Gaussian projection is a useful approximate export, not an exact change of file format.

3 Plain MRA integrals

The plain electronic Hamiltonian is rebuilt from the current MO trees in every orbital macro-iteration.

3.1 One-electron matrix

Let Ω denote the finite computational domain. The kinetic matrix starts in the Laplacian form

$$T_{pq} = -\frac{1}{2} \int_{\Omega} \phi_p^*(\mathbf{r}) \nabla^2 \phi_q(\mathbf{r}) \, d\mathbf{r}. \quad (\text{P1.14})$$

Green's first identity gives

$$T_{pq} = \frac{1}{2} \int_{\Omega} \nabla \phi_p^*(\mathbf{r}) \cdot \nabla \phi_q(\mathbf{r}) \, d\mathbf{r} - \frac{1}{2} \oint_{\partial\Omega} \phi_p^*(\mathbf{r}) \nabla \phi_q(\mathbf{r}) \cdot \mathbf{n} \, dS. \quad (\text{P1.15})$$

For orbitals that decay before the box boundary, or for boundary conditions that set this surface contribution to zero, the second term vanishes (or is numerically negligible). The kinetic energy can then be evaluated from the three Cartesian MRA derivative trees. This is why the one-electron implementation uses the gradient form even though the Hamiltonian contains $-\nabla^2/2$.

The local nuclear attraction has two useful forms:

$$V_{\text{nuc}}^{\text{point}}(\mathbf{r}) = - \sum_A \frac{Z_A}{|\mathbf{r} - \mathbf{R}_A|}, \quad (\text{P1.16})$$

$$V_{\text{nuc}}^{\text{smooth}}(\mathbf{r}) = - \sum_A \frac{Z_A}{c_A} u_{\text{sm}}\left(\frac{|\mathbf{r} - \mathbf{R}_A|}{c_A}\right),$$

$$u_{\text{sm}}(x) = \frac{\text{erf}(x)}{x} + \frac{e^{-x^2} + 16e^{-4x^2}}{3\sqrt{\pi}}, \quad c_A = \left(\frac{0.00435 \epsilon_{\text{nuc}}}{Z_A^5}\right)^{1/3}. \quad (\text{P1.17})$$

Equation (P1.17) is the Harrison–Fann–Yanai–Gan–Beylkin smooth nuclear potential [6]. Both the point Coulomb potential and this smooth regularization can be represented in MRA. As representative public workflows, an MRCPP SCF example uses the smooth form, while a VAMPyR introductory example constructs the point potential. The one-electron matrix is finally

$$h_{pq} = T_{pq} + \int_{\Omega} \phi_p^*(\mathbf{r}) V_{\text{nuc}}(\mathbf{r}) \phi_q(\mathbf{r}) \, d\mathbf{r}. \quad (\text{P1.18})$$

The following VAMPyR-style sketch assumes real orbital trees in one MRA world. The point or smooth nuclear potential is supplied as a Cartesian callable and projected once. Tree multiplication, cropping, differentiation, and contraction then use the public high-level operations demonstrated by VAMPyR [5].

Pseudocode: one-electron matrix

```
from vampyr import vampyr3d as vp
import numpy as np

# Phi[p]: real MRA tree for orbital p
project = vp.ScalingProjector(mra=mra, prec=prec)
V_nuc = project(v_nuc_callable)      # point or smooth model

D = vp.ABGVDerivative(mra=mra, a=0.0, b=0.0)
grad_phi = [vp.gradient(oper=D, inp=phi) for phi in Phi]
Vphi = [(V_nuc * phi).crop(prec) for phi in Phi]

norb = len(Phi)
h = np.empty((norb, norb))
for p in range(norb):
    for q in range(norb):
        Tpq = 0.5 * sum(
            vp.dot(grad_phi[p][axis], grad_phi[q][axis])
            for axis in range(3)
        )
        Vpq = vp.dot(Phi[p], Vphi[q])
        h[p, q] = Tpq + Vpq
```

Here `vp.gradient` returns the three Cartesian derivative trees and `vp.dot(tree_a, tree_b)` performs the final three-dimensional contraction. The kinetic expression is the gradient form of equation (P1.15); the code therefore does not construct a Laplacian tree. For complex orbitals, the bra trees must be conjugated explicitly. The current workflow uses real orbitals.

3.2 Two-electron tensor

For every orbital pair, form the MRA product

$$\rho_{pr}(\mathbf{r}) = \phi_p^*(\mathbf{r})\phi_r(\mathbf{r}). \quad (\text{P1.19})$$

The Coulomb field is a convolution of this pair density with the free-space Poisson Green function. To make the normalization unambiguous, define

$$G_0 = (-\nabla^2)^{-1}, \quad G_0(\mathbf{r}, \mathbf{r}') = \frac{1}{4\pi|\mathbf{r} - \mathbf{r}'|}, \quad (\text{P1.20})$$

because, in the distributional sense,

$$-\nabla_{\mathbf{r}}^2 \frac{1}{4\pi|\mathbf{r} - \mathbf{r}'|} = \delta(\mathbf{r} - \mathbf{r}'). \quad (\text{P1.21})$$

Consequently, the electrostatic potential of a pair density obeys

$$\begin{aligned} -\nabla^2 v_{qs}(\mathbf{r}) &= 4\pi\rho_{qs}(\mathbf{r}), \\ v_{qs} &= G_0[4\pi\rho_{qs}]. \end{aligned} \quad (\text{P1.22})$$

It is often convenient to name the Coulomb operator

$$C = 4\pi G_0, \quad C(\mathbf{r}, \mathbf{r}') = \frac{1}{|\mathbf{r} - \mathbf{r}'|}, \quad v_{qs} = C[\rho_{qs}]. \quad (\text{P1.23})$$

These are the same physical operation under two interfaces: an operator exposing G_0 must receive the source $4\pi\rho$, while an operator exposing C receives ρ . Passing ρ to G_0 , or $4\pi\rho$ to C , introduces an erroneous factor of 4π .

The four-index electron-repulsion integral is obtained by one further three-dimensional contraction:

$$g_{pqrs} = \int \rho_{pr}(\mathbf{r}_1) v_{qs}(\mathbf{r}_1) d\mathbf{r}_1 = \iint \frac{\phi_p^*(\mathbf{r}_1)\phi_q^*(\mathbf{r}_2)\phi_r(\mathbf{r}_1)\phi_s(\mathbf{r}_2)}{r_{12}} d\mathbf{r}_1 d\mathbf{r}_2. \quad (\text{P1.24})$$

Thus $g_{pqrs} = \langle pq|rs \rangle = (pr|qs)_{\text{chem}}$. A PySCF tensor in chemists' ordering is therefore mapped by $g_{pqrs}^{\text{phys}} = g_{prqs}^{\text{chem}}$. No six-dimensional orbital-pair grid is stored: multiplication produces a three-dimensional pair-density tree, convolution produces a three-dimensional potential tree, and the dot product produces one tensor element.

At the API level, pair-density trees and their Coulomb potentials can be cached before the four-index contraction. This avoids rebuilding ρ_{pr} or v_{qs} inside the innermost loop:

Pseudocode: two-electron tensor

```
from vampyr import vampyr3d as vp
import numpy as np

poisson = vp.PoissonOperator(mra=mra, prec=prec)
norb = len(Phi)
```

```

# Cache all real-orbital pair densities rho[p,r](r).
rho = {}
for p in range(norb):
    for r in range(norb):
        rho[p, r] = (Phi[p] * Phi[r]).crop(prec)

# Direct PoissonOperator call exposes G0, so its source is 4*pi*rho.
v_coul = {}
for q in range(norb):
    for s in range(norb):
        source = 4.0 * np.pi * rho[q, s]
        v_coul[q, s] = poisson(source).crop(prec)

# Physicists' ordering: g[p,q,r,s] = <rho[p,r] | v[q,s]>.
g = np.empty((norb, norb, norb, norb))
for p in range(norb):
    for q in range(norb):
        for r in range(norb):
            for s in range(norb):
                g[p, q, r, s] = vp.dot(rho[p, r], v_coul[q, s])

```

The final line is a three-dimensional tree contraction, not a six-dimensional quadrature. The explicit 4π matches equation (P1.22): `PoissonOperator` applies G_0 , whereas the Coulomb operator $C = 4\pi G_0$ acts directly on ρ . An implementation may retain only symmetry-unique orbital pairs, but the full loops above make the index mapping $g_{pqrs} = (pr|qs)_{\text{chem}}$ explicit.

4 MRA-DMRG orbital optimization

The numerical orbital optimization originates in the direct real-space Lagrangian method of Valeev *et al.* [1]. Nibbi *et al.* replace the correlated CI step by DMRG and obtain the required energy derivatives directly from the tensor-network calculation [2]. In operational terms, PySCF RHF MOs are first projected into MRA trees. Those trees generate h and g ; FCI or DMRG solves the finite second-quantized problem and returns reduced-density-matrix information; the orbital Lagrangian derivative generates new MRA trees. After orthonormalization, the integrals and many-body state are rebuilt. This alternating loop is repeated until the energy and orbital residuals converge.

The following is a code-independent restatement of Algorithm 1 of Nibbi *et al.* [2]; it is the sequential counterpart of figure 3.

4.1 Finite second-quantized problem

For fixed real orbitals, the finite-space Hamiltonian is written exactly in the convention of Nibbi *et al.* [2]:

$$\hat{H} = \sum_{ij\sigma} h_{ij} a_{i\sigma}^\dagger a_{j\sigma} + \frac{1}{2} \sum_{ijkl\sigma\tau} g_{ijkl} a_{i\sigma}^\dagger a_{j\tau}^\dagger a_{l\tau} a_{k\sigma}. \quad (\text{P1.25})$$

Here $g_{ijkl} = \langle ij|kl \rangle = (ik|jl)_{\text{chem}}$, so the annihilation order and the integral ordering must not be changed independently. Once h and g have been contracted from the trees, the many-body

Algorithm 1: MRA+DMRG orbital optimization, following Algorithm 1 of Ref. [2].

Input: Orthonormal MRA orbitals $\{\phi_m\}$, MRA precision p , and energy threshold δ

Output: Optimized orbitals $\{\phi_m\}$ and ground-state energy

```

1 while  $\Delta E > \delta$  do
2   Calculate  $h_{ij}$  and  $g_{ijkl}$  from the current orbital trees
3   Construct the MPO representation of  $\hat{H}$ 
4   Run DMRG to obtain  $E$  and the ground-state wavefunction
5   Extract  $\partial E / \partial h_{ij}$  and  $\partial E / \partial g_{ijkl}$ 
6   Compute the Lagrange-multiplier matrix  $\epsilon_{ij}$ 
7   Diagonalize  $\partial E / \partial h_{ij}$  and rotate the orbitals
8   Solve the real-space orbital-update equation with the bound-state Helmholtz Green
   operator
9   Apply Löwdin orthonormalization

```

solver is independent of the real-space representation. Its interface returns

$$\left(E_{\text{elec}}, \frac{\partial E}{\partial h}, \frac{\partial E}{\partial g} \right) = \text{solver}(h, g). \quad (\text{P1.26})$$

Following the paper’s spin-free coefficient convention, define

$$\begin{aligned} \hat{E}_{ij} &= \sum_{\sigma} a_{i\sigma}^{\dagger} a_{j\sigma}, \\ \hat{e}_{ijkl} &= \hat{E}_{ik} \hat{E}_{jl} - \delta_{jk} \hat{E}_{il} = \sum_{\sigma\tau} a_{i\sigma}^{\dagger} a_{j\tau}^{\dagger} a_{l\tau} a_{k\sigma}. \end{aligned} \quad (\text{P1.27})$$

The coefficient derivatives are

$$\frac{\partial E}{\partial h_{ij}} = \langle \hat{E}_{ij} \rangle \equiv \gamma_{ij}, \quad \frac{\partial E}{\partial g_{ijkl}} = \langle \hat{e}_{ijkl} \rangle \equiv \gamma_{ijkl}. \quad (\text{P1.28})$$

Equation (P1.28) deliberately reproduces the convention used in Ref. [2]; one must not insert an additional factor 1/2 when passing the returned, symmetry-aware molecular-Hamiltonian coefficient gradient into the orbital equations below.

The FCI backend is the exact oracle for small orbital spaces. The ChemTensor implementation [7] extracts $\partial E / \partial h$ and $\partial E / \partial g$ by differentiating the molecular MPO coefficients. The retained block2 backend [8] computes the equivalent objects from the one- and two-particle RDMs. These are alternative finite-space solvers; none of them changes how the MRA orbitals or integrals are represented.

4.2 Lagrangian derivative and orbital update

The Lagrangian derivative, orbital rotation, and Helmholtz update in this subsection follow Refs. [1, 2].

Orthonormality is imposed through

$$\mathcal{L} = E - \sum_{ij} \epsilon_{ij} (S_{ij} - \delta_{ij}), \quad S_{ij} = \langle \phi_i | \phi_j \rangle. \quad (\text{P1.29})$$

The orbital derivative follows the chain rule,

$$\frac{\partial E}{\partial \phi_m} = \sum_{ij} \frac{\partial h_{ij}}{\partial \phi_m} \frac{\partial E}{\partial h_{ij}} + \sum_{ijkl} \frac{\partial g_{ijkl}}{\partial \phi_m} \frac{\partial E}{\partial g_{ijkl}}. \quad (\text{P1.30})$$

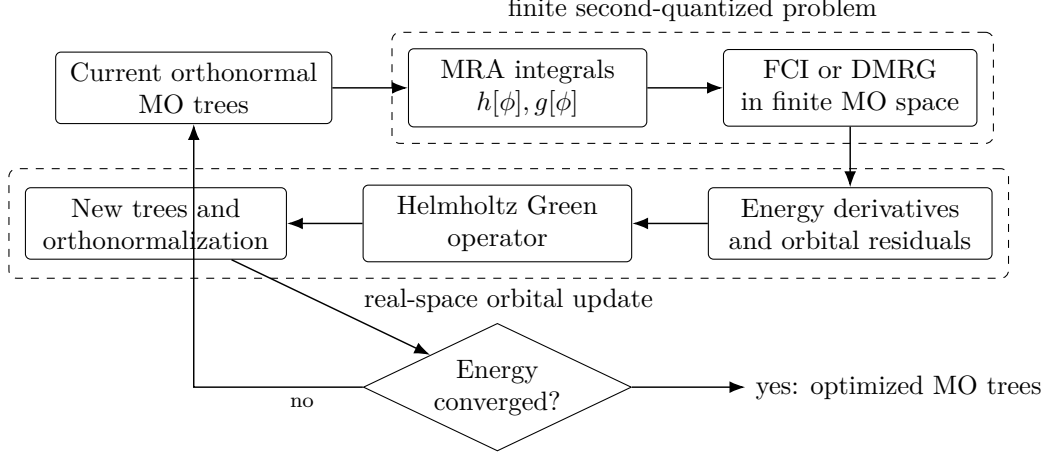


Figure 3: Separation between the finite many-body solve and the real-space orbital update.

For real orbitals, the combinations entering the functional derivative are

$$\partial E^{(1)} = \frac{1}{2} \left[\frac{\partial E}{\partial h} + \left(\frac{\partial E}{\partial h} \right)^T \right], \quad (\text{P1.31})$$

$$\partial E_{ijkl}^{(2)} = \frac{1}{4} \left[\frac{\partial E}{\partial g_{ijkl}} + \frac{\partial E}{\partial g_{jikl}} + \frac{\partial E}{\partial g_{kijl}} + \frac{\partial E}{\partial g_{lkij}} \right]. \quad (\text{P1.32})$$

The four terms in equation (P1.32) are the paper's $T_{0,1,2,3}$, $T_{1,0,3,2}$, $T_{2,1,0,3}$, and $T_{1,2,3,0}$ permutations written out explicitly. With $\hat{g}_{jl}f = v_{jl}f$, where v_{jl} is the Poisson field in equation (P1.20), stationarity gives

$$\sum_j \left(\partial E_{mj}^{(1)} \hat{h} - \bar{\epsilon}_{mj} \right) \phi_j + 2 \sum_{jkl} \partial E_{mjkl}^{(2)} \hat{g}_{jl} \phi_k = 0, \quad \bar{\epsilon} = \frac{1}{2}(\epsilon + \epsilon^T). \quad (\text{P1.33})$$

Projection onto the current orthonormal orbitals supplies the Lagrange multipliers:

$$\bar{\epsilon}_{mn} = \sum_j \partial E_{mj}^{(1)} h_{nj} + 2 \sum_{jkl} \partial E_{mjkl}^{(2)} g_{njl}. \quad (\text{P1.34})$$

The one-body derivative is diagonalized,

$$\partial E^{(1)} = U^\dagger \Lambda U, \quad \phi'_m = \sum_i U_{mi} \phi_i, \quad \epsilon' = U \bar{\epsilon} U^\dagger. \quad (\text{P1.35})$$

Splitting $\hat{h} = \hat{d} + \hat{v}$, with $\hat{d} = -\nabla^2/2$, gives the paper's differential working equation:

$$\begin{aligned} - \left(\hat{d} - \frac{\epsilon'_{mm}}{\Lambda_m} \right) \phi'_m &= \hat{v} \phi'_m \\ &+ \frac{1}{\Lambda_m} \left[2 \sum_{ijkl} U_{mi} \partial E_{ijkl}^{(2)} \hat{g}_{jl} \phi_k - \sum_{j \neq m} \epsilon'_{mj} \phi'_j \right]. \end{aligned} \quad (\text{P1.36})$$

Only this shifted differential equation is inverted with a bound-state Helmholtz Green operator:

$$\phi'_m = -G_{\epsilon'_{mm}/\Lambda_m} \left\{ \hat{v} \phi'_m + \frac{1}{\Lambda_m} \left[2 \sum_{ijkl} U_{mi} \partial E_{ijkl}^{(2)} \hat{g}_{jl} \phi_k - \sum_{j \neq m} \epsilon'_{mj} \phi'_j \right] \right\}, \quad \frac{\epsilon'_{mm}}{\Lambda_m} < 0. \quad (\text{P1.37})$$

The negative shift is required for a decaying bound-state kernel. All terms inside braces are assembled as MRA trees: nuclear action, Coulomb pair potentials weighted by $\partial E^{(2)}$, and off-diagonal multiplier couplings. The updated orbitals are cropped, Löwdin-orthonormalized, rebuilt into h and g , and iterated until the total-energy criterion is satisfied.

4.3 Output

The output is an orthonormal set of optimized *molecular orbitals* stored as adaptive MRA trees; it is not a numerical AO basis. Each tree can be evaluated, multiplied, differentiated, integrated, or acted on by an MRA convolution operator without first constructing a uniform grid. Part II uses the optimized MO trees, their pair densities, and the Cartesian MRA derivatives of the MOs. A Gaussian back-projection remains an optional approximate export.

Part II

Fixed-orbital erfc transcorrelation

5 Similarity transform and erfc Jastrow factor

For a fixed MO set, define the erfc pair Jastrow factor

$$u(r_{12}; \mu) = \frac{r_{12}}{2} \operatorname{erfc}(\mu r_{12}) - \frac{\exp[-(\mu r_{12})^2]}{2\sqrt{\pi}\mu}. \quad (\text{P2.1})$$

$$u'(r_{12}; \mu) = \frac{1}{2} \operatorname{erfc}(\mu r_{12}). \quad (\text{P2.2})$$

These expressions are the erfc correlation factor introduced by Giner [4]. With $\tau_\mu = \sum_{i<j} u(r_{ij}; \mu)$, the similarity-transformed Hamiltonian is

$$\tilde{H}(\mu) = e^{-\tau_\mu} \hat{H} e^{\tau_\mu} = \hat{H} - \sum_{i<j} \hat{K}_{ij}(\mu) - \sum_{i<j<k} \hat{L}_{ijk}(\mu). \quad (\text{P2.3})$$

The sign convention and the K/L decomposition follow Cohen *et al.* (arXiv:1908.02882) [3]. For one electron pair,

$$\begin{aligned} \hat{K}_{12} = & \underbrace{\nabla_1 u \cdot \nabla_1 + \nabla_2 u \cdot \nabla_2}_{\hat{K}_1} + \underbrace{\frac{1}{2}(\nabla_1^2 u + \nabla_2^2 u)}_{\hat{K}_2} \\ & + \underbrace{\frac{1}{2}(|\nabla_1 u|^2 + |\nabla_2 u|^2)}_{\hat{K}_3}. \end{aligned} \quad (\text{P2.4})$$

Because $u = u(r_{12})$, $\nabla_2 u = -\nabla_1 u$, $\nabla_2^2 u = \nabla_1^2 u$, and $|\nabla_2 u|^2 = |\nabla_1 u|^2$. Thus K_2 and K_3 reduce to one representative electron-1 matrix element, whereas the two drift derivatives in K_1 act on different ket orbitals and must both be kept. Cross terms between different pairs in $\frac{1}{2} \sum_i |\nabla_i \tau_\mu|^2$ generate the three-electron operator L .

In the same notation as Supplementary Eq. (S1) of Cohen *et al.*, the second-quantized Hamiltonian is

$$\begin{aligned} \tilde{H} = & \sum_{pq\sigma} h_q^p a_{p\sigma}^\dagger a_{q\sigma} + \frac{1}{2} \sum_{pqrs} (V_{rs}^{pq} - K_{rs}^{pq}) \sum_{\sigma\tau} a_{p\sigma}^\dagger a_{q\tau}^\dagger a_{s\tau} a_{r\sigma} \\ & - \frac{1}{6} \sum_{pqrstu} L_{stu}^{pqr} \sum_{\sigma\tau\lambda} a_{p\sigma}^\dagger a_{q\tau}^\dagger a_{r\lambda}^\dagger a_{u\lambda} a_{t\tau} a_{s\sigma}. \end{aligned} \quad (\text{P2.5})$$

Here

$$V_{rs}^{pq} = \langle \phi_p \phi_q | r_{12}^{-1} | \phi_r \phi_s \rangle = g_{pqrs}$$

is the ordinary physicists-ordered Coulomb tensor from Part I, and $K = K^{(1)} + K^{(2)} + K^{(3)}$. The corresponding Supplementary Eqs. (S2)–(S4) are

$$K_{rs}^{pq(1)} = \langle pq | \nabla_1 u(\mathbf{r}_1, \mathbf{r}_2) \cdot \nabla_1 | rs \rangle, \quad (\text{P2.6})$$

$$K_{rs}^{pq(2)} = \langle pq | \nabla_1^2 u(\mathbf{r}_1, \mathbf{r}_2) | rs \rangle, \quad (\text{P2.7})$$

$$K_{rs}^{pq(3)} = \langle pq | |\nabla_1 u(\mathbf{r}_1, \mathbf{r}_2)|^2 | rs \rangle. \quad (\text{P2.8})$$

The tensor entering the two-body part is therefore

$$g_{\text{TC}} = V - K^{(1)} - K^{(2)} - K^{(3)}. \quad (\text{P2.9})$$

All displayed V and K tensors use physicists' ordering; conversion to chemists' ordering swaps the middle indices as in Part I. For helium, $\hat{L}$ vanishes exactly because only two electrons are present.

6 Radial kernels and Gaussian separation

The three radial functions required by $\hat{K}_{12}$ are

$$q_1(r) = \frac{u'(r)}{r} = \frac{\text{erfc}(\mu r)}{2r}, \quad (\text{P2.10})$$

$$q_2(r) = \nabla_1^2 u(r) = \frac{\text{erfc}(\mu r)}{r} - \frac{\mu}{\sqrt{\pi}} e^{-\mu^2 r^2}, \quad (\text{P2.11})$$

$$q_3(r) = \frac{|\nabla_1 u(r)|^2}{r^2} = \frac{\text{erfc}^2(\mu r)}{4r^2}. \quad (\text{P2.12})$$

They enter through

$$\nabla_1 u = (\mathbf{r}_1 - \mathbf{r}_2) q_1(r_{12}), \quad \nabla_1^2 u = q_2(r_{12}), \quad |\nabla_1 u|^2 = r_{12}^2 q_3(r_{12}).$$

These equations are analytic identities. The approximation sign first enters when a radial kernel is replaced by a finite Gaussian sum for the current custom-convolution interface:

$$q_a(r) \simeq \sum_{m=1}^{M_a} c_{am} e^{-\zeta_{am} r^2}, \quad a = 1, 2, 3. \quad (\text{P2.13})$$

The orbitals, pair densities, and intermediate fields are not fitted to Gaussians; they remain adaptive MRA trees. Both q_1 and q_3 are singular auxiliary radial factors at $r = 0$. The operators use the physical combinations $(\mathbf{r}_1 - \mathbf{r}_2) q_1$ and $r^2 q_3 = |\nabla u|^2$. A finite Gaussian sum resolves these combinations over a specified radial interval; it does not reproduce q_1 or q_3 pointwise at the single coalescence point.

6.1 Giner's Gaussian-orbital route and the present MRA route

The Gaussian decomposition in the present MRA implementation is not the 20-term fit used by Giner [4]. In Giner's Gaussian-orbital route, the drift and Laplacian factors containing $\text{erfc}(\mu r)/r$ are evaluated with analytic range-separated Gaussian integrals. The squared-erfc integral in K_3 is identified as the non-analytic two-electron case and is approximated in two stages:

$$\text{erfc}(x) \simeq e^{-\alpha x - \beta x^2}, \quad e^{-X} \simeq \sum_{m=1}^{20} c_m e^{-\zeta_m X^2}. \quad (\text{P2.14})$$

With $\alpha = 1.09529$ and $\beta = 0.756023$, this gives the plain isotropic Gaussian representation

$$\frac{1}{4} \text{erfc}^2(\mu r) \simeq \frac{1}{4} \sum_{m=1}^{20} c_m e^{-z_m r^2}, \quad z_m = (2\alpha\mu)^2 \zeta_m + 2\beta\mu^2. \quad (\text{P2.15})$$

The exponent in equation (P2.15) follows directly from the two preceding fit equations. Thus Giner's fit is a scalar power-zero representation of K_3 ; it is not applied to K_1 and K_2 in that Gaussian-orbital route.

Table 1: Gaussian representations in Giner’s Gaussian-orbital route and in the present MRA convolution route.

Term	Giner Gaussian-orbital route	Present MRA convolution route
K_1	Analytic integrals with the $\text{erfc}(\mu r)/r$ drift factor.	Finite GL Gaussian sum for $\text{erfc}(\mu r)/r$, followed by three power-one convolutions.
K_2	Analytic range-separated Coulomb and ordinary Gaussian integrals.	The same finite GL sum plus the explicit negative Gaussian, applied as a signed scalar convolution.
K_3	Fitted 20-term plain isotropic Gaussian sum for $\text{erfc}^2(\mu r)$.	Finite GL discretization of an exact positive integral representation, followed by three power-two convolutions.

6.2 Exact integral identities and finite GL sums

The present construction starts from the exact identity

$$\frac{\text{erfc}(\mu r)}{r} = \frac{2}{\sqrt{\pi}} \int_{\mu}^{\infty} e^{-s^2 r^2} ds, \quad (\text{P2.16})$$

and the exact positive representation

$$\text{erfc}^2(\mu r) = \frac{4r^2}{\pi} \int_{\sqrt{2}\mu}^{\infty} q \left[\frac{\pi}{2} - 2 \arcsin\left(\frac{\mu}{q}\right) \right] e^{-q^2 r^2} dq. \quad (\text{P2.17})$$

The first numerical approximation is the replacement of each infinite upper limit by $4/r_{\min}$. Logarithmic substitutions then map the retained intervals to

$$s = \mu e^t, \quad 0 \leq t \leq T_1 = \log\left(\frac{4/r_{\min}}{\mu}\right), \quad (\text{P2.18})$$

$$q = \sqrt{2}\mu e^{\tau}, \quad 0 \leq \tau \leq T_3 = \log\left(\frac{4/r_{\min}}{\sqrt{2}\mu}\right). \quad (\text{P2.19})$$

Let $\{\xi_m, w_m\}_{m=1}^M$ be the Gauss–Legendre nodes and weights on $[-1, 1]$. For equation (P2.18), define

$$t_m = \frac{T_1}{2}(\xi_m + 1), \quad \omega_m = \frac{T_1}{2}w_m, \quad s_m = \mu e^{t_m}.$$

The finite sum for $\text{erfc}(\mu r)/r$ has

$$c_m^{(1)} = \frac{2}{\sqrt{\pi}} s_m \omega_m, \quad \zeta_m^{(1)} = s_m^2. \quad (\text{P2.20})$$

For equation (P2.19), define

$$\tau_m = \frac{T_3}{2}(\xi_m + 1), \quad \tilde{\omega}_m = \frac{T_3}{2}w_m, \quad q_m = \sqrt{2}\mu e^{\tau_m}.$$

The corresponding finite representation is

$$\text{erfc}^2(\mu r) \simeq r^2 \sum_{m=1}^M c_m^{(3)} e^{-\zeta_m^{(3)} r^2}, \quad (\text{P2.21})$$

$$c_m^{(3)} = \frac{4}{\pi} q_m^2 \left[\frac{\pi}{2} - 2 \arcsin\left(\frac{\mu}{q_m}\right) \right] \tilde{\omega}_m, \quad \zeta_m^{(3)} = q_m^2. \quad (\text{P2.22})$$

All coefficients in equations (P2.20) and (P2.22) are non-negative. The q_1 coefficients are one half of $c_m^{(1)}$. The q_2 sum uses $c_m^{(1)}$ and adds the exact negative Gaussian $-(\mu/\sqrt{\pi})e^{-\mu^2 r^2}$. The q_3 coefficients are one quarter of $c_m^{(3)}$.

The production construction uses

$$x_{\min} = \mu r_{\min} = 8.7 \times 10^{-5}, \quad M = 24 \quad (\text{GL24 production default}). \quad (\text{P2.23})$$

GL32 and GL64 use the same equations with 32 and 64 nodes. They provide progressively stricter calculations and kernel-convergence checks, and still higher orders can be used when required. Increasing M reduces the quadrature-discretization error but increases the separated rank and the operator cost. It does not by itself remove the finite-upper-limit error or the independent MRA representation error. GL order, radial cutoff, and MRA precision must therefore be converged separately. Kernel tests compare each finite sum with the analytic radial function and with higher-order constructions.

Table 2: Exact and approximate layers in the present TC construction.

Layer	Status
Definitions of u, u', q_1, q_2, q_3	Analytic and exact
Infinite-integral identities for erfc/r and erfc^2	Analytic and exact
Replacement of the infinite upper limit by $4/r_{\min}$	Numerical approximation
GL24, GL32, or GL64 finite Gaussian sum	Numerical approximation
Cartesian separation and $r^2 = x^2 + y^2 + z^2$	Algebraically exact
MRA convolution, compression, and cropping	Numerical, controlled by the MRA precision
Final TC tensor	Contains kernel-representation and MRA-truncation errors

6.3 Why the present MRA route uses Gaussian sums

After tree-space orbital optimization, a molecular orbital is a general adaptive multiwavelet tree; it is not, in general, a finite expansion in the starting Gaussian AO basis. Gaussian-orbital integral recurrences are therefore no longer a representation-independent route to the TC integrals. The calculation instead constructs an MRA pair density, applies a translation-invariant convolution kernel, and contracts the resulting three-dimensional MRA field.

For a custom kernel, the present MRCPP/VAMPyR convolution interface accepts a finite Gaussian expansion through `GaussExp` and applies it with `ConvolutionOperator` or `CartesianConvolution`. This current API is the reason that both $\text{erfc}(\mu r)/r$ and $\text{erfc}^2(\mu r)$ are converted to finite Gaussian sums in the MRA route. It is an implementation compromise, not a mathematical requirement of the erfc Jastrow or of MRA theory. A native radial operator or a different separated representation could replace this layer without changing the analytic TC equations.

$$\begin{aligned}
 &\text{general MRA molecular orbital} \implies \text{GTO recurrences no longer apply} \\
 \implies &\text{MRA convolution} \implies \text{current custom operator requires a finite Gaussian sum} \\
 &\implies \text{finite Gauss-Legendre quadrature}
 \end{aligned}$$

(P2.24)

A Gaussian kernel is separable in Cartesian relative coordinates:

$$e^{-\zeta|\mathbf{r}_1-\mathbf{r}_2|^2} = e^{-\zeta(x_1-x_2)^2} e^{-\zeta(y_1-y_2)^2} e^{-\zeta(z_1-z_2)^2}. \quad (\text{P2.25})$$

The current Cartesian convolution operator applies each separated term directly to a pair-density tree. The two-electron dependence enters through $\mathbf{r}_1 - \mathbf{r}_2$; an explicit six-dimensional grid is never constructed.

7 Operator-level construction of K_1 , K_2 , K_3 , and L

Let $\rho_{qs}(\mathbf{r}) = \phi_q^*(\mathbf{r})\phi_s(\mathbf{r})$. Each TC tensor is built from three-dimensional fields generated by convolving this pair density with the appropriate relative-coordinate kernel. The following equations show precisely which field is formed and how it is contracted.

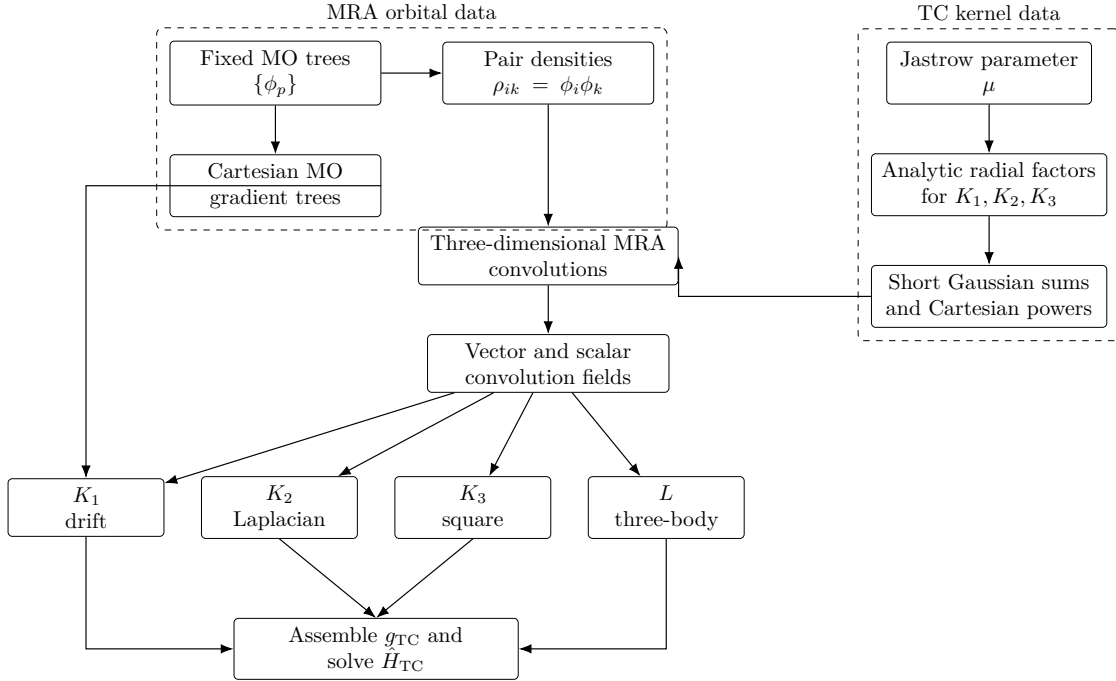


Figure 4: TC integral construction. Gaussian decomposition is applied to radial kernels only; the orbitals and all intermediate fields remain MRA trees.

7.1 K_1 : non-Hermitian drift term

For electron 1, define the vector field

$$A_{qs}^\alpha(\mathbf{r}_1) = \int \rho_{qs}(\mathbf{r}_2)(r_{1\alpha} - r_{2\alpha})q_1(r_{12}) \, d\mathbf{r}_2, \quad \alpha \in \{x, y, z\}. \quad (\text{P2.26})$$

The representative drift integral in equation (P2.6) then becomes

$$K_{rs}^{pq(1a)} = \sum_{\alpha=x,y,z} \int \phi_p^*(\mathbf{r}_1) A_{qs}^\alpha(\mathbf{r}_1) \partial_\alpha \phi_r(\mathbf{r}_1) \, d\mathbf{r}_1. \quad (\text{P2.27})$$

The electron-2 derivative is the exchanged contribution,

$$K_{rs}^{pq(1b)} = K_{sr}^{qp(1a)}, \quad K_{rs}^{pq(1)} = K_{rs}^{pq(1a)} + K_{rs}^{pq(1b)}. \quad (\text{P2.28})$$

This term is non-Hermitian because the derivative acts on the ket orbital. The only numerical differentiation here is $\partial_\alpha \phi_r$, obtained by applying the MRA derivative operator to the MO tree. The Jastrow derivative is analytic; its radial factor is represented numerically by the finite GL sum described above.

For one Gaussian term in q_1 , the x component factors exactly as

$$(x_1 - x_2)e^{-\zeta r_{12}^2} = \left[(x_1 - x_2)e^{-\zeta(x_1-x_2)^2} \right] \times e^{-\zeta(y_1-y_2)^2} e^{-\zeta(z_1-z_2)^2}. \quad (\text{P2.29})$$

The y and z components are cyclic permutations. Thus each Gaussian term generates three separable convolutions: power one along one Cartesian axis and power zero along the other two. Their sum over m produces the three MRA trees $A_{qs}^x, A_{qs}^y, A_{qs}^z$. The finite radial sum is an approximation; the power-one factorization of every retained Gaussian is an exact algebraic identity.

7.2 K_2 : direct Laplacian convolution

Define the scalar convolution field

$$B_{qs}(\mathbf{r}_1) = \int q_2(r_{12}) \rho_{qs}(\mathbf{r}_2) \, d\mathbf{r}_2. \quad (\text{P2.30})$$

Then

$$K_{rs}^{pq(2)} = \int \rho_{pr}(\mathbf{r}_1) B_{qs}(\mathbf{r}_1) \, d\mathbf{r}_1. \quad (\text{P2.31})$$

The $\text{erfc}(\mu r)/r$ part of q_2 is represented by the finite GL sum, whereas $-(\mu/\sqrt{\pi})e^{-\mu^2 r^2}$ is already one exact Gaussian at the kernel level. The current convolution backend does not safely retain negative Gaussian coefficients. The implementation therefore applies separate positive and absolute-negative power-zero operators and subtracts their output fields. This sign split is an API workaround, not a change to the analytic definition of q_2 .

The atom-centered-grid route of Cohen *et al.* uses integration by parts to avoid a difficult direct numerical Laplacian integral. Under a vanishing surface term, the same identity is

$$K_{rs}^{pq(2)} = - \iint \nabla_1 \rho_{pr}(\mathbf{r}_1) \cdot \nabla_1 u(r_{12}) \rho_{qs}(\mathbf{r}_2) \, d\mathbf{r}_1 \, d\mathbf{r}_2. \quad (\text{P2.32})$$

In the present formulation $\nabla_1^2 u = q_2$ is known analytically as a relative-coordinate kernel. Its finite Gaussian representation can therefore be applied directly as in equation (P2.31), without numerically differentiating u . Equation (P2.32) is an equivalent identity, not the required production formula.

7.3 K_3 : squared-gradient scalar term

Define

$$D_{qs}(\mathbf{r}_1) = \int r_{12}^2 q_3(r_{12}) \rho_{qs}(\mathbf{r}_2) \, d\mathbf{r}_2. \quad (\text{P2.33})$$

The scalar tensor is

$$K_{rs}^{pq(3)} = \int \rho_{pr}(\mathbf{r}_1) D_{qs}(\mathbf{r}_1) \, d\mathbf{r}_1. \quad (\text{P2.34})$$

Discretizing the exact positive representation of erfc^2 gives the finite Gaussian approximation to q_3 ,

$$r_{12}^2 q_3(r_{12}) \simeq \sum_m c_{3m} [(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2] e^{-\zeta_{3m} r_{12}^2}. \quad (\text{P2.35})$$

For example,

$$(x_1 - x_2)^2 e^{-\zeta r_{12}^2} = \left[(x_1 - x_2)^2 e^{-\zeta(x_1 - x_2)^2} \right] \times e^{-\zeta(y_1 - y_2)^2} e^{-\zeta(z_1 - z_2)^2}. \quad (\text{P2.36})$$

Therefore each Gaussian term becomes the sum of three separable power-two convolutions, one for each Cartesian axis. The finite GL sum is approximate, while $r^2 = x^2 + y^2 + z^2$ and each Cartesian factorization are exact. This differs from Giner's fitted power-zero scalar route. No MO derivative is used in K_3 .

7.4 VAMPyR-style bridge from Gaussian sums to MRA fields

The following non-normative pseudocode illustrates the current API mapping. The equations above define the operators; the class and method names only show how the present MRCP/ VAMPyR interface realizes them. The expensive operator objects are built once for each μ and reused over all pair densities.

Listing 1: VAMPyR-style illustration of the present TC convolution bridge.

```
gaussians -> GaussFunc -> GaussExp

K1:
    drift = CartesianConvolution(mra, drift_terms, precision)
    for powers in [(1,0,0), (0,1,0), (0,0,1)]:
        drift.setCartesianComponents(*powers)
        A[powers] = crop(drift(pair_density), precision)

K2:
    pos = ConvolutionOperator(mra, positive_terms, precision)
    neg = ConvolutionOperator(mra, abs(negative_terms), precision)
    B = crop(pos(pair_density) - neg(pair_density), precision)

K3:
    square = CartesianConvolution(mra, square_terms, precision)
    D = 0
    for powers in [(2,0,0), (0,2,0), (0,0,2)]:
        square.setCartesianComponents(*powers)
        D += square(pair_density)
    D = crop(D, precision)
```

7.5 L : factored three-electron term

For systems with three or more electrons, the vector fields $\mathbf{A}_{qs}$ from K_1 are reused. In the notation of Cohen *et al.*, a factored six-index matrix element has the structure

$$L_{stu}^{pqr} = \int \rho_{ps} \mathbf{A}_{qt} \cdot \mathbf{A}_{ru} \, d\mathbf{r} + \int \rho_{qt} \mathbf{A}_{ps} \cdot \mathbf{A}_{ru} \, d\mathbf{r} + \int \rho_{ru} \mathbf{A}_{ps} \cdot \mathbf{A}_{qt} \, d\mathbf{r}. \quad (\text{P2.37})$$

The products are three-dimensional scalar MRA fields. The full dense six-index tensor need not be materialized. For helium, this section is skipped exactly.

In summary, K_1 is a vector power-one convolution followed by contraction with MO derivative trees; K_2 is a signed scalar convolution with the analytic Laplacian kernel; K_3 is the sum of three scalar power-two convolutions; and L reuses the K_1 vector fields. All orbital and intermediate fields remain adaptive MRA trees.

8 Capabilities and boundaries

The complete conceptual pipeline is now:

Gaussian RHF MOs $\longrightarrow$ MRA MO trees $\longrightarrow$ plain $h, g \longrightarrow$ FCI/DMRG optimization
 $\longrightarrow$ optimized MRA MO trees $\longrightarrow$ fixed-orbital erfc-TC.

The first optimization stage follows the direct real-space formulation of Valeev *et al.* (arXiv:2207.10841) and the MRA-DMRG extension of Nibbi *et al.* (arXiv:2503.10808v3) [1, 2]. ChemTensor and block2 are alternative finite-space solvers behind the same conceptual interface [7, 8]. The second stage follows the TC decomposition of Cohen *et al.* (arXiv:1908.02882) and the erfc Jastrow factor of Giner (arXiv:2101.09944) [3, 4].

For a fixed MO family, the pair densities, plain integrals, Coulomb fields, and MO derivative trees are independent of μ . Changing μ changes the radial Gaussian sums and the resulting TC convolution fields. For helium, $L = 0$ exactly; for systems with three or more electrons the three-body term must be retained or replaced by a separately justified approximation.

The optimized MRA orbitals can be evaluated at arbitrary Cartesian coordinates and differentiated directly in the adaptive representation. They can also be projected onto a chosen Gaussian AO basis, but that export is approximate and may require a substantially enlarged Gaussian space. The MRA tree plus a Cartesian orbital evaluator therefore preserves the native numerical information most directly.

Two boundaries are important. First, the orbital optimization described in Part I uses the plain Hamiltonian; the TC Hamiltonian is added afterward at fixed orbitals. Second, the erfc parameter μ is scanned rather than optimized in the present study. Neither restriction is imposed by the MRA representation itself.

References

- [1] E. F. Valeev, R. J. Harrison, A. A. Holmes, C. C. Peterson, and D. A. Penchoff. “Direct Determination of Optimal Real-Space Orbitals for Correlated Electronic Structure of Molecules”. In: *Journal of Chemical Theory and Computation* 19.20 (2023), pp. 7230–7241. DOI: 10.1021/acs.jctc.3c00732. arXiv: 2207.10841.
- [2] M. Nibbi, L. Frediani, E. Dinvey, and C. B. Mendl. *Wavefunction Optimization at the Complete Basis Set Limit with Multiwavelets and DMRG*. Version 3. 2025. arXiv: 2503.10808 [physics.chem-ph].
- [3] A. J. Cohen, H. Luo, K. Guthier, W. Dobrautz, D. P. Tew, and A. Alavi. “Similarity Transformation of the Electronic Schrödinger Equation via Jastrow Factorization”. In: *The Journal of Chemical Physics* 151.6 (2019), p. 061101. DOI: 10.1063/1.5116024. arXiv: 1908.02882.
- [4] E. Giner. “A New Form of Transcorrelated Hamiltonian Inspired by Range-Separated DFT”. In: *The Journal of Chemical Physics* 154.8 (2021), p. 084119. DOI: 10.1063/5.0044683. arXiv: 2101.09944.
- [5] M. Bjørgve, C. Tantardini, S. R. Jensen, G. A. Gerez S., P. Wind, R. Di Remigio Eikås, E. Dinvey, and L. Frediani. “VAMPyR: A High-Level Python Library for Mathematical Operations in a Multiwavelet Representation”. In: *The Journal of Chemical Physics* 160.16 (2024), p. 162502. DOI: 10.1063/5.0203401.
- [6] R. J. Harrison, G. I. Fann, T. Yanai, Z. Gan, and G. Beylkin. “Multiresolution Quantum Chemistry: Basic Theory and Initial Applications”. In: *The Journal of Chemical Physics* 121.23 (2004), pp. 11587–11598. DOI: 10.1063/1.1791051.
- [7] C. B. Mendl and M. Reumann. *ChemTensor*. Tensor-network library. 2025. URL: <https://github.com/qc-tum/chemtensor>.
- [8] H. Zhai, H. R. Larsson, S. Lee, Z.-H. Cui, T. Zhu, C. Sun, L. Peng, R. Peng, K. Liao, J. Tölle, J. Yang, S. Li, and G. K.-L. Chan. “Block2: A Comprehensive Open Source Framework to Develop and Apply State-of-the-Art DMRG Algorithms in Electronic Structure and Beyond”. In: *The Journal of Chemical Physics* 159.23 (2023), p. 234801. DOI: 10.1063/5.0180424.